

# Chapitre 5 : Générateurs et décorateurs

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Fil rouge** : le registre des colis de Yonnee Express · **Version** : 2026-10-23

## Fiche du chapitre

| | |
|---|---|
| **Objectifs** | A la fin du chapitre, l'apprenant sait : 1) expliquer la différence entre un **itérable** et un **itérateur**, et ce que fait `next` ; 2) écrire une fonction **génératrice** avec `yield` ; 3) choisir entre une liste et une **expression génératrice**, et mesurer la mémoire utilisée ; 4) lire un très gros fichier ligne par ligne, et enchaîner des générateurs en **pipeline** ; 5) utiliser `islice`, `count` et `chain` du module `itertools` ; 6) écrire un **décorateur**, avec `functools.wraps` ; 7) écrire un décorateur **avec paramètre** ; 8) accélérer une fonction avec `functools.lru_cache`, après avoir **mesuré**. |
| **Prérequis** | Chapitres 1 à 4, en particulier les fonctions en paramètre et les fermetures (chapitre 4, section 6) |
| **Durée** | 🟢 Débutant : 7 h · 🔵 Standard : 5 h · 🟣 Expert : 3 h 30 (atelier compris) |
| **Matériel** | Python 3.12 ou plus récent (vérifié avec Python 3.13), VS Code ; le script `fabriquer_grand_registre.py` qui vous est fourni |
| **Livrable** | L'atelier 5 : 500 000 livraisons traitées sans saturer la mémoire, publié sur GitHub |

### Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Créez un dossier `essais-ch5`. Copiez-y le script `fabriquer_grand_registre.py` et lancez-le une fois :

```
python fabriquer_grand_registre.py
```

```
registre_grand.csv : 500 000 livraisons écrites
```

Le fichier `registre_grand.csv` (environ 25 Mo) apparaît dans le dossier : trop gros pour être partagé sur Discord, chacun le fabrique sur son ordinateur. La graine du hasard est fixe : tout le monde obtient **exactement** le même fichier, donc les mêmes résultats que ce chapitre.

---

## 1. Le problème : 500 000 livraisons

Yonnee Express a grandi : le registre compte maintenant **500 000** livraisons. La méthode des chapitres précédents (tout charger dans une liste, puis calculer) marche encore, mais elle range en mémoire 500 000 dictionnaires d'un coup. Sur l'ordinateur de la direction, ou sur un petit serveur loué, c'est la différence entre un programme qui tourne et un programme qui bloque la machine.

Ce chapitre apprend à traiter les données **une par une**, au moment où on en a besoin : c'est le rôle des **générateurs**. Puis il apprend à ajouter un comportement (chronométrer, journaliser, mémoriser) à n'importe quelle fonction **sans la modifier** : c'est le rôle des **décorateurs**.

## 2. Itérables et itérateurs

Une boucle `for` sait parcourir une liste, un texte, un dictionnaire, un fichier. Comment fait-elle ? Créez `essai1.py` :

```
quartiers = ["Médina", "Yoff", "Pikine"]
iterateur = iter(quartiers)
print(next(iterateur))
print(next(iterateur))
print(next(iterateur))
try:
    next(iterateur)
except StopIteration:
    print("Plus rien : StopIteration")
```

```
Médina
Yoff
Pikine
Plus rien : StopIteration
```

| Morceau | Ce qu'il veut dire |
|---|---|
| `quartiers` | Un **itérable** : un objet qu'on peut parcourir (liste, texte, dictionnaire, fichier…) |
| `iter(quartiers)` | Demande à la liste un **itérateur** : un objet qui sait où il en est du parcours |
| `next(iterateur)` | « Donne-moi l'élément suivant » ; l'itérateur avance d'une case |
| `StopIteration` | L'exception levée par `next` quand il n'y a plus rien |

🟢 Une image : la liste est un **livre**, l'itérateur est un **marque-page**. Le livre ne change pas ; le marque-page avance à chaque `next`.

Une boucle `for` fait exactement cela en cachette : `for q in quartiers:` appelle `iter(quartiers)`, puis `next` à chaque tour, et s'arrête proprement quand elle reçoit `StopIteration`. Retenez ce mécanisme : un générateur est un itérateur qu'on écrit soi-même.

## 3. Les générateurs : `yield`

### 3.1 Une fonction qui produit plusieurs valeurs

Créez `essai2.py` :

```
def numeros_de_suivi(debut, nombre):
    for n in range(debut, debut + nombre):
        yield f"YX-{n:06d}"


generateur = numeros_de_suivi(41, 3)
print(generateur)
print(next(generateur))
for numero in generateur:
    print(numero)
print(list(generateur))
```

```
<generator object numeros_de_suivi at 0x00000209E2D0C2E0>
YX-000041
YX-000042
YX-000043
[]
```

| Morceau | Ce qu'il veut dire |
|---|---|
| `yield f"YX-{n:06d}"` | `yield` (« produire ») **donne** une valeur et **met la fonction en pause** à cet endroit. Une fonction qui contient `yield` est une **fonction génératrice** |
| `{n:06d}` | Le nombre sur 6 chiffres, complété par des zéros à gauche : `41` devient `000041` |
| `numeros_de_suivi(41, 3)` | L'appel **n'exécute rien** : il renvoie un **générateur**, prêt à démarrer. L'adresse `0x...` change à chaque exécution |
| `next(generateur)` | La fonction démarre, va jusqu'au premier `yield`, donne `YX-000041` et se met en pause |
| `for numero in generateur` | La boucle continue **là où on s'était arrêté** : 42, puis 43 |
| `list(generateur)` | Vide : un générateur ne se parcourt qu'**une seule fois**. Pour recommencer, il faut en créer un nouveau en rappelant la fonction |

> 🧨 **Essayez** : ajoutez une dernière ligne `next(generateur)`. Le programme s'arrête sur `StopIteration`, sans autre message : le générateur est épuisé. Annulez.

### 3.2 La pause, pas à pas

🟢 Pour voir l'ordre réel des opérations, créez `essai3.py` :

```
def compte_a_rebours(n):
    print("début")
    while n > 0:
        print("  je produis", n)
        yield n
        n -= 1
    print("fin")


for valeur in compte_a_rebours(3):
    print("reçu", valeur)
```

```
début
  je produis 3
reçu 3
  je produis 2
reçu 2
  je produis 1
reçu 1
fin
```

Les lignes **alternent** : le générateur produit 3, la boucle le reçoit et l'affiche, **puis seulement** le générateur reprend après son `yield`, fait `n -= 1`, et produit 2. Les valeurs ne sont jamais toutes calculées à l'avance : chacune est fabriquée **quand la boucle la demande**. C'est ce qu'on appelle l'**évaluation paresseuse** (*lazy evaluation*).

Quand la fonction arrive à sa fin (`print("fin")`), le générateur lève `StopIteration` tout seul, et la boucle s'arrête.

| Une fonction normale | Une fonction génératrice |
|---|---|
| `return` donne **une** valeur et **termine** la fonction | `yield` donne une valeur et **met en pause** ; on peut en donner autant qu'on veut |
| L'appel exécute le corps tout de suite | L'appel renvoie un générateur ; le corps s'exécute au fil des `next` |
| Ses variables disparaissent à la fin | Ses variables (`n` ici) sont **conservées** entre deux `yield` |

## 4. Les expressions génératrices et la mémoire

Au chapitre 4, vous avez écrit des compréhensions de listes entre crochets. Avec des **parenthèses**, on obtient une **expression génératrice** : la même chose, en paresseux. Créez `essai4.py` :

```python
import sys

carres_liste = [n * n for n in range(1_000_000)]
carres_generateur = (n * n for n in range(1_000_000))
print(f"liste : {sys.getsizeof(carres_liste):,} octets".replace(",", " "))
print(f"générateur : {sys.getsizeof(carres_generateur)} octets")
print(sum(carres_generateur))
print(sum(carres_generateur))
```

```
liste : 8 448 728 octets
générateur : 200 octets
333332833333500000
0
```

| Morceau | Ce qu'il veut dire |
|---|---|
| `[n * n for n in ...]` | Une liste : le million de carrés est calculé et rangé **tout de suite** |
| `(n * n for n in ...)` | Une expression génératrice : **rien** n'est calculé ; les carrés le seront un par un |
| `sys.getsizeof(...)` | La taille de l'objet en mémoire, en octets |
| `1_000_000` | Un million ; les `_` ne servent qu'à la lisibilité |
| `{...:,}` puis `.replace(",", " ")` | Les milliers séparés par une virgule, remplacée par une espace (écriture française) |
| Le deuxième `sum` donne 0 | Le générateur a été épuisé par le premier `sum` : il ne produit plus rien |

La liste occupe **8,4 Mo** (et encore : `getsizeof` ne compte que le tableau de cases, pas les nombres eux-mêmes). Le générateur occupe **200 octets**, qu'il produise un million de valeurs ou un milliard.

Quand une expression génératrice est le seul argument d'une fonction, on peut enlever ses parenthèses : `sum(n * n for n in range(1_000_000))`. C'est l'écriture la plus courante.

> 🧨 **Essayez** : sur le générateur, `len(carres_generateur)` puis `carres_generateur[0]`. Messages : `TypeError: object of type 'generator' has no len()` (« un générateur n'a pas de longueur ») et `TypeError: 'generator' object is not subscriptable` (« on ne peut pas prendre un élément par sa position »). Logique : il ne connaît ni son nombre d'éléments ni ses éléments à l'avance. Annulez.

### Quand choisir quoi ?

| Choisir une liste si… | Choisir un générateur si… |
|---|---|
| on parcourt les données **plusieurs fois** | on les parcourt **une seule fois** (une somme, un comptage, un export) |
| on a besoin de `len`, de `[i]`, de trier | les données sont **très nombreuses**, voire sans fin |
| les données sont peu nombreuses | on veut s'arrêter **dès** qu'on a trouvé ce qu'on cherche |

## 5. Lire un très gros fichier

### 5.1 Liste contre générateur, mesuré

Voici la vraie question de Yonnee Express : le total des frais des 500 000 livraisons. Deux méthodes, et on **mesure** la mémoire. Créez `essai5.py` dans le dossier du grand registre :

```
import csv
import time
import tracemalloc


def lire_livraisons(chemin):
    with open(chemin, newline="", encoding="utf-8") as fichier:
        for ligne in csv.DictReader(fichier, delimiter=";"):
            yield ligne


def mesurer(fonction):
    tracemalloc.start()
    debut = time.perf_counter()
    resultat = fonction()
    duree = time.perf_counter() - debut
    _, pic = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return resultat, duree, pic / 1_000_000


def frais_avec_liste():
    with open("registre_grand.csv", newline="", encoding="utf-8") as fichier:
        toutes = list(csv.DictReader(fichier, delimiter=";"))
    return sum(int(l["frais"]) for l in toutes)


def frais_avec_generateur():
    return sum(int(l["frais"]) for l in lire_livraisons("registre_grand.csv"))


for nom, fonction in [("liste", frais_avec_liste), ("générateur", frais_avec_generateur)]:
    total, duree, pic = mesurer(fonction)
    total_lisible = f"{total:,}".replace(",", " ")
    print(f"{nom:11} total {total_lisible} F ; {duree:.1f} s ; mémoire maximale {pic:.1f} Mo")
```

```
liste       total 1 048 561 000 F ; 9.9 s ; mémoire maximale 326.4 Mo
générateur  total 1 048 561 000 F ; 7.7 s ; mémoire maximale 0.1 Mo
```

| Morceau | Ce qu'il veut dire |
|---|---|
| `lire_livraisons` | Une fonction génératrice : elle ouvre le fichier et donne les lignes **une par une** |

| Morceau | Ce qu'il veut dire |
|---|---|
| `with open(...)` dans le générateur | Le fichier reste ouvert tant que le générateur tourne, et se ferme quand il est épuisé |
| `tracemalloc.start()` | Commence à surveiller la mémoire utilisée par Python (*trace memory allocations*) |
| `tracemalloc.get_traced_memory()` | Renvoie deux nombres : la mémoire utilisée maintenant et le **pic** (le maximum) depuis `start`. `_, pic = ...` garde le second ; `_` est le nom habituel d'une valeur qu'on ignore |
| `time.perf_counter()` | Un chronomètre précis ; la différence entre deux appels donne la durée en secondes |
| `mesurer(fonction)` | On passe la fonction **sans parenthèses** (chapitre 4) ; `mesurer` l'appelle entre le début et la fin des mesures |
| `{nom:11}` | Le nom sur 11 caractères, pour aligner les colonnes |

Même total, mais **326 Mo** contre **0,1 Mo** : 3 000 fois moins de mémoire. Avec la liste, les 500 000 dictionnaires existent tous en même temps ; avec le générateur, il n'y en a jamais qu'**un** à la fois : il est lu, additionné, puis oublié.

Les durées dépendent de l'ordinateur (entre 8 et 15 secondes ici, d'une exécution à l'autre) ; `tracemalloc` ralentit beaucoup les deux versions. Sans lui, chacune prend moins de 2 secondes. Ce qui compte, c'est l'écart de mémoire : il ne varie pas.

🟣 Pourquoi 326 Mo pour un fichier de 25 Mo ? Un dictionnaire Python stocke, pour chaque ligne, 8 clés et 8 textes, chacun étant un objet avec son propre en-tête. Les données occupent en mémoire **plus de dix fois** la taille du fichier : c'est fréquent, et c'est pour cela qu'un fichier « pas si gros » peut saturer un ordinateur.

## 5.2 Des générateurs en chaîne : le pipeline

Les générateurs se branchent les uns sur les autres, comme des tuyaux : chacun prend les éléments du précédent, les transforme ou les trie, et les passe au suivant. Créez `essai6.py` :

```
import csv
from collections import Counter


def lire_livraisons(chemin):
    with open(chemin, newline="", encoding="utf-8") as fichier:
        yield from csv.DictReader(fichier, delimiter=";")


def convertir(lignes):
    for l in lignes:
        yield {"quartier": l["quartier"], "duree": int(l["duree_min"]), "frais": int(l["frais"])}


def en_retard(livraisons, seuil=60):
    return (l for l in livraisons if l["duree"] > seuil)


retards = Counter(l["quartier"] for l in
         en_retard(convertir(lire_livraisons("registre_grand.csv"))))
print(retards.most_common())
```

```
[('Pikine', 27458), ('Parcelles', 13661), ('Yoff', 5582), ('Plateau', 1712), ('Médina', 458)]
```

| Morceau | Ce qu'il veut dire |
|---|---|
| `yield from csv.DictReader(...)` | « Donne, un par un, tous les éléments de cet itérable. » Raccourci de `for ligne in ...: yield ligne` |
| `convertir(lignes)` | Reçoit un itérable de lignes de texte, produit des dictionnaires avec des nombres |
| `en_retard(livraisons, seuil=60)` | Renvoie une expression génératrice : seulement les livraisons de plus de 60 minutes |
| `en_retard(convertir(lire_livraisons(...)))` | Le **pipeline** : lire, puis convertir, puis filtrer |
| `Counter(...)` | Le compteur du chapitre 3 ; c'est lui qui « tire » les livraisons dans le tuyau |

Lisez le pipeline de l'intérieur vers l'extérieur. Rien ne se passe tant que `Counter` ne demande pas d'élément : il demande une livraison en retard à `en_retard`, qui en demande une convertie à `convertir`, qui demande une ligne à `lire_livraisons`, qui la lit dans le fichier. Une seule livraison traverse le tuyau à la fois, quelle que soit la taille du fichier.

Chaque étape est une petite fonction, facile à tester seule et à réutiliser : c'est le même esprit que les filtres combinables de l'atelier 4, en version paresseuse.

## 6. Le module `itertools`

`itertools` contient des outils tout faits pour les itérateurs. Trois sont très utiles. Créez `essai7.py` :

```python
import csv
from itertools import chain, count, islice


def lire_livraisons(chemin):
    with open(chemin, newline="", encoding="utf-8") as fichier:
        yield from csv.DictReader(fichier, delimiter=";")


for ligne in islice(lire_livraisons("registre_grand.csv"), 3):
    print(ligne["numero"], ligne["date"], ligne["quartier"])

numeros = count(start=1001)
print(next(numeros), next(numeros))
print(list(chain(["Médina", "Yoff"], ("Pikine",))))
```

```
1 2025-04-16 Médina
2 2026-07-15 Parcelles
3 2025-04-11 Plateau
1001 1002
['Médina', 'Yoff', 'Pikine']
```

| Outil | Ce qu'il fait |
|---|---|
| `islice(iterable, 3)` | Les 3 premiers éléments, comme `[:3]` sur une liste, mais pour n'importe quel itérateur. Le fichier n'est lu que sur 3 lignes, pas sur 500 000 ; on peut aussi écrire `islice(it, debut, fin)` |
| `count(start=1001)` | Compte sans fin : 1001, 1002, 1003… Idéal pour numéroter. Sans fin : ne jamais faire `list(count())` |
| `chain(a, b)` | Enchaîne plusieurs itérables comme s'ils n'en formaient qu'un, sans les copier |

> 🖍️ **Essayez** : `print(lire_livraisons("registre_grand.csv")[:3])` . Message : `TypeError: 'generator' object is not subscriptable` . Les crochets ne marchent que sur les listes, les textes, les tuples ; pour un générateur, c'est `islice` . Annulez.

## 7. Les décorateurs

### 7.1 Le besoin

Vous voulez savoir combien de temps prend chacune de vos fonctions de traitement. Ajouter `debut = time.perf_counter()` au début et un `print` à la fin de **chaque** fonction ? C'est répétitif, ça mélange la mesure et le traitement, et il faudra tout retirer ensuite. Un **décorateur** ajoute ce comportement autour d'une fonction, sans toucher à son code.

Il repose sur deux idées du chapitre 4 : une fonction est une **valeur** (on peut la passer et la renvoyer), et une fonction intérieure **se souvient** des variables de la fonction englobante (fermeture).

## 7.2 Un décorateur chronomètre

Créez `essai8.py` :

```
import time
from functools import wraps


def chronometre(fonction):
    @wraps(fonction)
    def enveloppe(*args, **kwargs):
        debut = time.perf_counter()
        resultat = fonction(*args, **kwargs)
        print(f"[{fonction.__name__}] {time.perf_counter() - debut:.3f} s")
        return resultat
    return enveloppe


@chronometre
def somme_des_carres(n):
    return sum(k * k for k in range(n))


print(somme_des_carres(1_000_000))
print(somme_des_carres.__name__)
```

```
[somme_des_carres] 0.090 s
333332833333500000
somme_des_carres
```

| Morceau | Ce qu'il veut dire |
|---|---|
| `def chronometre(fonction)` | Le décorateur : une fonction qui **reçoit une fonction**… |
| `def enveloppe(*args, **kwargs)` | … fabrique une nouvelle fonction qui l'**enveloppe**. `*args, **kwargs` (chapitre 4) : elle accepte n'importe quels arguments |
| `resultat = fonction(*args, **kwargs)` | Elle appelle la fonction d'origine avec les mêmes arguments, et garde son résultat |
| `print(...)` puis `return resultat` | Elle ajoute son travail (afficher la durée) et renvoie le résultat **sans le changer** |
| `return enveloppe` | Le décorateur renvoie l'enveloppe (sans parenthèses) |
| `@chronometre` au-dessus de `def` | Raccourci de `somme_des_carres = chronometre(somme_des_carres)` : le nom désigne désormais l'enveloppe |
| `fonction.__name__` | Le nom de la fonction, en texte |
| `@wraps(fonction)` | Copie sur l'enveloppe le nom et la docstring de la fonction d'origine |

🟢 Lisez l'appel `somme_des_carres(1_000_000)` dans l'ordre : on appelle en réalité `enveloppe(1_000_000)` ; elle démarre le chronomètre, appelle la vraie `somme_des_carres(1_000_000)`, affiche la durée, renvoie le résultat. Le code de `somme_des_carres` n'a pas changé d'une ligne ; pour arrêter de chronométrer, on supprime la ligne `@chronometre`.

> 🖍️ **Essayez** : 1) retirez la ligne `@wraps(fonction)` : la dernière ligne affiche `enveloppe` au lieu de `somme_des_carres`. Toutes vos fonctions chronométrées s'appelleraient « enveloppe » dans les messages d'erreur : d'où `wraps`. 2) Remettez-la, puis supprimez `return enveloppe`. Message, à l'appel : `TypeError: 'NoneType' object is not callable` (« `None` ne peut pas être appelé ») : sans `return`, le décorateur renvoie `None`, et `somme_des_carres` vaut `None`. Annulez.

### 7.3 Un décorateur avec un paramètre

On veut parfois régler le décorateur : `@journaliser("[FRAIS]")`. Il faut alors **un étage de plus** : une fonction qui reçoit le réglage et **fabrique** le décorateur. Créez `essai9.py` :

```
from functools import wraps


def journaliser(etiquette):
    def decorateur(fonction):
        @wraps(fonction)
        def enveloppe(*args, **kwargs):
            print(f"{etiquette} appel de {fonction.__name__}{args}")
            return fonction(*args, **kwargs)
        return enveloppe
    return decorateur


@journaliser("[FRAIS]")
def frais(quartier, poids):
    return {"Médina": 1000, "Pikine": 2500}[quartier] + (500 if poids > 5 else 0)


print(frais("Pikine", 7))
```

```
[FRAIS] appel de frais('Pikine', 7)
3000
```

| Étage | Reçoit | Renvoie |
|---|---|---|
| `journaliser(etiquette)` | le réglage, `"[FRAIS]"` | `decorateur` |
| `decorateur(fonction)` | la fonction `frais` | `enveloppe` |
| `enveloppe(*args, **kwargs)` | les arguments de l'appel, `("Pikine", 7)` | le résultat de `frais` |

`@journaliser("[FRAIS]")` se lit en deux temps : d'abord `journaliser("[FRAIS]")` est appelé et renvoie `decorateur` ; puis `decorateur` est appliqué à `frais`. L'enveloppe se souvient à la fois de `etiquette` et de `fonction` : deux fermetures emboîtées. `{args}` affiche le tuple des arguments : `('Pikine', 7)`.

🟣 On peut empiler plusieurs décorateurs : `@chronometre` puis `@journaliser("[FRAIS]")` sur deux lignes. Le plus proche du `def` s'applique en premier.

## 8. Mémoriser les résultats : `lru_cache`

Certaines fonctions sont appelées très souvent avec les **mêmes** arguments. Plutôt que de recalculer, on peut garder les résultats en mémoire : c'est un **cache**. Python en fournit un sous forme de décorateur. Créez `essai10.py` :

```
import time
from functools import lru_cache


def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)


@lru_cache(maxsize=None)
def fibonacci_cache(n):
    if n < 2:
        return n
    return fibonacci_cache(n - 1) + fibonacci_cache(n - 2)


debut = time.perf_counter()
print(fibonacci(30), f"{time.perf_counter() - debut:.2f} s sans cache")
debut = time.perf_counter()
print(fibonacci_cache(30), f"{time.perf_counter() - debut:.5f} s avec cache")
print(fibonacci_cache.cache_info())
print(fibonacci_cache(200))
```

```
832040 0.18 s sans cache
832040 0.00007 s avec cache
CacheInfo(hits=28, misses=31, maxsize=None, currsize=31)
280571172992510140037611932413038677189525
```

| Morceau | Ce qu'il veut dire |
|---|---|
| `fibonacci` | La suite de Fibonacci (chaque nombre est la somme des deux précédents), en récursif (chapitre 4). `fibonacci(30)` fait 2,7 millions d'appels, en recalculant sans cesse les mêmes valeurs |

| Morceau | Ce qu'il veut dire |
|---|---|
| `@lru_cache(maxsize=None)` | Avant de calculer, le décorateur regarde s'il a déjà la réponse pour ces arguments ; sinon il calcule et la range. `maxsize=None` : pas de limite de taille. LRU (*least recently used*) : avec une limite, il oublie d'abord les réponses les moins récemment utilisées |
| `cache_info()` | `misses` : 31 calculs réels (de 0 à 30) ; `hits` : 28 réponses trouvées dans le cache ; `currsize` : 31 réponses gardées |
| `fibonacci_cache(200)` | Instantané avec le cache ; sans lui, l'univers aurait le temps de s'éteindre avant la fin du calcul |

Plus de **2 000 fois** plus rapide, sans changer une ligne du calcul.

**Deux conditions** pour qu'un cache soit utile et juste :

1. La fonction donne **toujours le même résultat** pour les mêmes arguments, et ne fait rien d'autre (pas d'affichage, pas d'écriture dans un fichier). Une fonction qui lit l'heure ou un fichier ne doit pas être mise en cache.
2. Les **mêmes arguments reviennent souvent**. Sinon, le cache ne fait que consommer de la mémoire.

> 🧨 **Essayez** : `@lru_cache(maxsize=None)` sur `def total(liste): return sum(liste)`, puis `total([1, 2])`. Message : `TypeError: unhashable type: 'list'`. Le cache range les arguments comme les clés d'un dictionnaire (chapitre 3) : une liste, modifiable, ne peut pas être une clé. Avec un tuple, `total((1, 2))` marche et donne 3. Annulez.

**Mesurer avant d'optimiser.** Un cache n'accélère que ce qui est **lent**. Dans l'atelier, vous comparerez trois façons de convertir 500 000 dates : vous découvrirez qu'un cache peut rendre une fonction lente 40 fois plus rapide, et qu'il ne sert à rien sur une fonction déjà rapide. On ne devine pas ce qui est lent : on le **mesure**, avec le décorateur chronomètre par exemple.

## Exercices

Créez un dossier `exercices-ch5`.

- ★ **Exercice 1 : deux générateurs.** Écrivez `plage(debut, fin, pas)`, une fonction génératrice qui fait comme `range` mais accepte des nombres à virgule : `list(plage(0, 1, 0.25))` donne `[0, 0.25, 0.5, 0.75]`, et `list(plage(10, 20, 3))` donne `[10, 13, 16, 19]`. Puis `fibonacci()`, un générateur **sans fin** des nombres de Fibonacci (0, 1, 1, 2, 3, 5…) ; affichez les 10 premiers avec `islice`. (Sections 3 et 6.)
- ★ **Exercice 2 : expressions génératrices.** En une ligne chacune : la somme des carrés des nombres impairs inférieurs à 1 000 (attendu : 166 666 500) ; la taille, avec `sys.getsizeof`, de la liste puis du générateur des carrés des impairs jusqu'à un million ; « existe-t-il un nombre de 0 à un million dont le carré dépasse 500 000 ? » avec `any` ; le **premier** nombre entier dont le carré dépasse 500 000, avec `next` et `count(1)` (attendu : 708). (Sections 4 et 6.)

- ★★ **Exercice 3 : les nombres premiers.** Écrivez `est_premier(n)` (indice : `all(n % d != 0 for d in range(2, int(n ** 0.5) + 1))`, et 0 et 1 ne sont pas premiers), puis `premiers()`, un générateur sans fin des nombres premiers. Affichez les 10 premiers ; le premier nombre premier au-delà de 10 000 ; parmi les 1 000 premiers nombres premiers, combien se terminent par 7 ? (Sections 3 et 6.)
- ★★ **Exercice 4 : deux décorateurs.** Écrivez `compter_appels`, un décorateur qui compte les appels de la fonction dans un attribut `appels` de l'enveloppe (indice : `enveloppe.appels = 0` avant le `return enveloppe`, et `enveloppe.appels += 1` dedans). Testez sur `saluer(nom)`, qui renvoie `"Bonjour " + nom`, appelée 3 fois. Puis `repeter(fois)`, un décorateur avec paramètre qui exécute la fonction `fois` fois et renvoie le dernier résultat ; testez `@repeter(3)` sur une fonction qui affiche un mot en majuscules et renvoie sa longueur. Vérifiez que `__name__` est conservé. (Section 7.)
- ★★★ **Exercice 5 : l'escalier.** De combien de façons peut-on monter un escalier de `n` marches, en faisant des pas de 1, 2 ou 3 marches ? Écrivez la fonction récursive `escalier(n)` (cas de base : 1 façon pour 0 marche, 0 façon pour un nombre négatif). Attendu : `escalier(4)` vaut 7. Avec votre décorateur `compter_appels`, combien d'appels pour `escalier(20)` ? Écrivez ensuite votre propre décorateur `memoriser`, avec un dictionnaire, et comptez à nouveau les appels. Comparez enfin avec `lru_cache` pour `n = 60`. (Sections 7 et 8.)

## Atelier 5 : 500 000 livraisons sans saturer la mémoire

Le travail principal du chapitre : vous reprenez le grand registre et construisez `grand_registre.py`, un outil qui calcule les indicateurs de la direction en **une seule lecture** du fichier, mesure lui-même sa mémoire et sa durée grâce à vos décorateurs, et exporte un extrait des retards sans lire tout le fichier. L'énoncé vous est fourni séparément.

## A retenir

- Un **itérable** se parcourt ; un **itérateur** sait où il en est ; `next` donne l'élément suivant, `StopIteration` signale la fin. Une boucle `for` fait tout cela en cachette.
- Une fonction qui contient `yield` est **génératrice** : l'appel ne fait rien, il renvoie un générateur ; chaque `next` exécute jusqu'au `yield` suivant, puis met en **pause**.
- Un générateur ne se parcourt qu'**une fois**, n'a ni `len` ni `[i]`, mais n'occupe presque **aucune mémoire**.
- `(... for ... in ...)` : expression génératrice ; `yield from iterable` : donner tous les éléments d'un itérable.
- Les gros fichiers se lisent **ligne par ligne** avec un générateur, et se traitent en **pipeline** de petits générateurs.
- `itertools` : `islice` (les premiers éléments, et on s'arrête de lire), `count` (compter sans fin), `chain` (enchaîner).
- Un **décorateur** reçoit une fonction et renvoie une **enveloppe** qui l'appelle ; `@deco` au-dessus du `def` ; toujours `@wraps(fonction)` ; avec paramètre, un étage de plus.
- `@lru_cache` mémorise les résultats d'une fonction **sans effet de bord** dont les arguments **reviennent souvent** ; arguments non modifiables seulement. **Mesurer avant d'optimiser.**

## Erreurs fréquentes

| Message ou symptôme | Ce que ça veut dire | Solution |
|---|---|---|
| `StopIteration` | `next` sur un itérateur épuisé | Utiliser une boucle `for`, ou `next(it, valeur_par_defaut)` |
| Un deuxième parcours ne donne rien (`[]`, une somme de 0) | Le générateur a déjà été épuisé | Rappeler la fonction génératrice, ou utiliser une liste si on parcourt plusieurs fois |
| `TypeError: object of type 'generator' has no len()` | `len` sur un générateur | Compter avec `sum(1 for _ in gen)`, ou faire une liste |
| `TypeError: 'generator' object is not subscriptable` | `gen[0]` ou `gen[:3]` | `next(gen)` ou `islice(gen, 3)` |
| La fonction ne fait rien quand on l'appelle | Elle contient `yield` : c'est une fonction génératrice | La parcourir (`for`, `list`, `next`) |
| Le programme ne s'arrête jamais | `list(...)` ou `for` sans `break` sur un générateur sans fin (`count`) | `islice`, ou `next(...)` avec une condition |
| `TypeError: 'NoneType' object is not callable` après `@deco` | Le décorateur ne renvoie pas l'enveloppe | `return enveloppe` à la fin du décorateur |
| Toutes les fonctions décorées s'appellent `enveloppe` | `@wraps` oublié | `@wraps(fonction)` au-dessus de `def enveloppe` |
| La fonction décorée renvoie `None` | L'enveloppe ne renvoie pas le résultat | `return fonction(*args, **kwargs)` (ou `return resultat`) |
| `TypeError: unhashable type: 'list'` avec `lru_cache` | Argument modifiable | Passer un tuple à la place d'une liste |

## Pour aller plus loin

- Le tutoriel officiel, sections « Générateurs » et « Expressions génératrices » (en français) : https://docs.python.org/fr/3/tutorial/classes.html#generators
- La documentation du module `itertools` : https://docs.python.org/fr/3/library/itertools.html
- La documentation de `functools` (`wraps`, `lru_cache`, `cache`) : https://docs.python.org/fr/3/library/functools.html
- Le guide de programmation fonctionnelle de Python : https://docs.python.org/fr/3/howto/functional.html